

Spring MVC

Servlet 기반 웹 MVC와 DispatcherServlet 요청 흐름

작성 기준: 원문 페이지를 확인한 뒤 만든 학습용 한국어 번역 요약본입니다. 공식문서 전체 문장을 그대로 복제한 전문 직역본은 아니며, 원문 URL을 함께 남겼습니다.

[1] 전체 구조 설명

- Spring Web MVC는 Servlet API 위에서 만들어진 Spring의 원래 웹 프레임워크다.
- Spring MVC는 Spring WebFlux와 병렬로 존재한다. MVC는 Servlet 기반, WebFlux는 reactive stack 기반이다.
- Spring MVC 요청 흐름의 중심은 DispatcherServlet이다.
- DispatcherServlet은 front controller 패턴을 사용해 요청 처리 알고리즘을 공유하고, 실제 작업은 여러 delegate component에 맡긴다.

[2] 문서별 직역체 학습 정리

Spring Web MVC

- Spring Web MVC는 spring-webmvc 모듈 이름에서 공식 명칭이 왔고, 보통 Spring MVC라고 부른다.
- 이 문서는 Servlet stack 기반 웹 애플리케이션을 다룬다. reactive stack이 필요하면 WebFlux 문서를 봐야 한다.
- Spring MVC 장은 DispatcherServlet, Filter, HTTP message conversion, annotated controller, functional endpoint, CORS, error response, view technology, MVC config 등을 다룬다.
- 게시판 과제에서는 Controller가 HTTP 요청을 받고 Service로 위임하는 구조가 Spring MVC 위에서 동작한다.

DispatcherServlet

- DispatcherServlet은 중앙 Servlet으로서 요청 처리의 공통 절차를 담당한다.
- 실제 요청 매핑, 뷰 해석, 예외 처리 등은 설정 가능한 delegate component가 수행한다.
- DispatcherServlet도 Servlet이므로 Java 설정 또는 web.xml을 통해 선언되고 URL에 매핑되어야 한다.
- Spring Boot에서는 대부분 자동 설정이 DispatcherServlet 등록을 처리하므로, 개발자는 Controller와 handler method 작성에 집중한다.
- 요청은 DispatcherServlet으로 들어온 뒤 HandlerMapping, HandlerAdapter, Controller, ViewResolver 또는 HTTP message converter 같은 구성요소를 거쳐 응답으로 나간다.

[3] 핵심 실행 흐름

HTTP 요청 도착 → Servlet container가 DispatcherServlet 호출 → HandlerMapping이 적절한 controller method 탐색 → HandlerAdapter가 method 실행 → Controller가 Service 호출 후 결과 반환 → JSON 변환 또는 view 해석 → HTTP 응답 반환

[4] 중요한 포인트 정리

- Controller는 비즈니스 로직을 직접 처리하지 않고 요청/응답 경계 역할을 맡는 것이 좋다.
- REST API에서는 ViewResolver보다 HttpMessageConverter가 더 중요하다.
- DispatcherServlet을 이해하면 Spring MVC의 요청 흐름이 Controller 중심이 아니라 front controller 중심이라는 점이 보인다.

원문 URL

<https://docs.spring.io/spring-framework/reference/web/webmvc.html>

<https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-servlet.html>